

GAN

Generative Adversarial Nets

1. Introduction

배경

딥러닝 목표: 복잡한 데이터(이미지, 음성, 텍스트)를 잘 표현할 수 있는 계층적 모델을 만드는 것



1. 계층적 모델?

- 말 그대로 여러 layer가 있다는 뜻
- 각 단계는 데이터를 더 복잡하고 추상적인 방식으로 표현

ex:

계층(층)	학습하는 정보
1층	선, 모서리 같은 아주 단순한 특징
2층	눈, 코, 입 같은 기본적인 패턴
3층	얼굴 전체 같은 복합적인 구조

→ 단계별로 복잡도를 높여가면서 데이터의 본질을 점점 잘 파악하게 되는 구조

2. 왜 계층적으로?

복잡한 데이터(사진, 음성, 언어)를 바로 한 번에 이해하려고 하면 어려움

⇒ 단순한 것부터 점점 복잡한 것까지 나눠서 학습하면 효율적!

3. 어디에 쓰이냐면

- **CNN(합성곱 신경망)**: 이미지 처리
- **RNN/LSTM**: 문장 구조나 시간 흐름 계층적으로 처리
- **Transformer**: 단어 간 관계를 여러 단계로 쌓아서 의미 파악

그동안 분류 문제에 특화된 판별 모델(discriminative model)에서 큰 성공

by 백프로퍼게이션(backpropagation) & 드롭아웃(dropout) 같은 기법 + ReLU 같은 선형 활성화 함수

하지만 생성 모델(generative model) 분야로는 잘 이어지지 않음

생성 모델이 어려웠던 이유

1. **최대우도추정(MLE)**: 학습 과정에서 수식이 복잡하고 계산 시간 오래
2. **비선형 함수**: 생성 모델 구조에서는 잘 맞지 않음



MLE가 생성 모델에서 어려운 이유

1. 우도 계산 어려움

생성 모델에서는 latent variable 포함하는 경우가 많음

이럴 때 진짜 데이터 x 가 나올 확률:

$$p(x) = \int p(x | z) \cdot p(z) dz$$

→ z (latent variable)에 대해 **적분**해야 하는데 보통 계산 불가능 or 너무 느려서 쓸 수 없음

2. 정확한 우도값 없이도 샘플은 잘 뽑히는 경우

MLE: 이 x 가 얼마나 가능성 있는 데이터인가를 수치로 표현
but 생성 모델은 그냥 샘플만 잘 뽑으면 됨
→ 우도는 계산하기 어렵고, 실제로는 꼭 필요하지도 않다!

3. MLE는 훈련 안정적으로 x

MLE 보통 로짓(logit) 기반 확률 사용
→ 모델이 틀린 예측 하면 손실 엄청 커지거나, 그래디언트가 사라지거나 폭발
특히 이미지처럼 고차원 데이터는 확률 정확하게 추정해야 해서 오히려 훈련 불안정해지고 잘 안 됨

GAN 등장

적대적 생성 네트워크(Generative Adversarial Network, GAN)

두 개의 모델을 서로 경쟁시키면서 동시에 훈련

- **Generator (G)**: 무작위 입력값으로부터 가짜 데이터를 만들. 목표는 진짜처럼 보이게 하는 것
- **Discriminator (D)**: 입력된 데이터가 진짜인지, G가 만든 가짜인지 판별

이 두 모델은 서로 맞붙어 게임처럼 경쟁하면서 더 정교하게 학습!!

장점

- G와 D 모두 다층 퍼셉트론(MLP)으로 만들 수 있음
- 학습은 기존 딥러닝처럼 backpropagation과 dropout만으로도 충분
- 추론 과정이나 MCMC 같은 복잡한 계산이 필요 x
- 샘플 만들 때 G에 무작위 벡터 z 만 넣으면 forward 연산만으로 새로운 데이터를 얻을 수 있음

2. Related work

2.1 그래프 기반 생성 모델

(1) 제한된 볼츠만 머신(RBM), 딥 볼츠만 머신(DBM)

이 모델들은 확률 분포를 명시적으로 정의함

but 이 분포를 계산하려면 모든 경우의 수를 합쳐야 되는데, 이게 거의 불가능

예: $p(x) = \frac{1}{Z} \prod \phi(x, h)$ (Z 는 전체 상태의 합)

→ 그래서 MCMC라는 느리고 복잡한 샘플링 기법을 써야 했음

(2) 딥 빌리프 네트워크(DBN)

RBM과 신경망을 섞은 형태

여전히 계산 복잡, 학습도 단계별로 따로 해야 해서 불편

2.2 로그 우도 없이 학습

(1) Score Matching, Noise-Contrastive Estimation (NCE)

확률 모델 직접 학습 어렵거나 정규화 상수 알 수 없을 때, “진짜 vs 가짜(노이즈)”를 분류하는 문제로 바꿔서 학습하는 방법

확률 직접 계산 안 하고 확률 밀도 함수의 모양만 맞춤

NCE는 GAN과 비슷하게 가짜와 진짜를 구분하는 판별 기준을 쓴다.

* but NCE는 노이즈 분포(판별 시 사용)를 고정해서 학습이 느려지고 전체 변수에 대해서 작동 X



NCE는 노이즈 분포 $q(x)$ 를 학습 중에 계속 바꾸는 게 아니라 **처음부터 하나 고정해놓고 끝까지 사용**
ex: 정규분포, 균등분포...

왜 문제?

1. 모델이 조금만 좋아져도 학습이 느려짐

모델이 어느 정도 잘 학습되면, 노이즈 샘플과 진짜 샘플의 구분이 너무 쉬워짐
→ 이러면 loss가 거의 0, gradient도 사라지고, 학습 속도 감소

2. 모든 변수 공간에 대해서 잘 작동하지 않음

고차원일수록 노이즈 분포가 진짜 데이터 분포랑 겹치는 영역이 거의 없음
모델은 이걸 노이즈라고 맞히기만 하니까 진짜 데이터가 어떤 모양인지 학습 X
⇒ 전체 입력공간(변수 공간)에 걸쳐 분포를 잘 배우기 어렵다

2.3 확률 분포 없이 샘플만 만드는 방식

(1) Denoising Autoencoder, GSN (Generative Stochastic Networks)

노이즈가 섞인 데이터를 원래대로 복원하면서 간접적으로 샘플링 배움

피드백 구조(반복적인 연산)가 필요해서 느리고 불안정

(2) VAE (Variational Autoencoder), Stochastic Backpropagation

확률 분포를 근사 추론해서 학습

여전히 추론 네트워크를 따로 훈련해야 하고, 수학적으로 복잡한 부분이 많음.

기존 방식	한계점
확률 기반 모델	복잡한 정규화 상수 계산 필요, MCMC 샘플링 필요
NCE	고정된 노이즈 분포 사용 → 학습 제한
Autoencoder 계열	피드백 구조 필요 → 느림
VAE 등	추론 네트워크 필요, 수식 복잡

GAN은 다 필요 없다:

- 확률 분포 명시 안 함
- 추론 네트워크 없음
- 단순히 G와 D만 훈련하면 됨
- forward만으로 샘플링 가능

3. Adversarial Nets

1. GAN 기본 구조

GAN은 두 개의 신경망을 이용해 학습한다:

- **Generator** $G(z; \theta_g)$
: 무작위 노이즈 $z \sim p_z(z)$ 를 받아서 진짜 같은 데이터 x 만들어냄
- **Discriminator** $D(x; \theta_d)$
: 입력된 데이터 x 가 진짜 데이터인지 G가 만든 가짜 데이터인지 판별

2. 학습

G와 D가 서로 경쟁하는 minimax 게임 통해 학습

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

항	의미
$\log D(\mathbf{x})$	진짜 데이터 맞힐 확률 (D가 잘함)
$\log(1 - D(G(\mathbf{z})))$	가짜 데이터를 가짜라고 맞힐 확률 (D가 잘함 → G 안 좋음)

- **D**: 진짜는 1에 가깝게, 가짜는 0에 가깝게 분류하려고 함
- **G**: D를 속이기 위해 자기가 만든 $G(\mathbf{z})$ 진짜처럼 보이게 하려 함

관점	진짜 데이터를 진짜라고 맞히게 → $\log D(\mathbf{x})$	가짜 데이터를 가짜라고 맞히게 → $\log(1 - D(G(\mathbf{z})))$
Discriminator 입장	둘 다 학습함 (진짜=1, 가짜=0로)	둘 다 중요
Generator 입장	해당 없음 (G는 진짜 안 만짐)	여기만 해당 : D가 자기가 만든 가짜를 어떻게 보는지가 중요함

학습 진행:

1. D 업데이트 (k번 반복)

- 진짜 데이터 $\mathbf{x} \sim p_{\text{data}}$
- G가 만든 가짜 $G(\mathbf{z}), \mathbf{z} \sim p_{\mathbf{z}}$ 넣고

D를 학습하여 진짜/가짜를 잘 구분하도록 만듦

2. G 업데이트 (1번)

- **$D(G(\mathbf{z}))$ 값을 최대화하는 방향으로 G 업데이트**

D가 G의 출력을 착각하게 만드는 방향으로 학습

⚠ 논문에서는:

위 공식에선 G 학습할 때 $\log(1 - D(G(\mathbf{z})))$ 를 **최소화**하는 방향

근데 이러면 초기에 $D(G(\mathbf{z})) \approx 0$ 이라 gradient가 **너무 작아짐** → 학습이 안 됨



G의 학습 목표: $\min_G \log(1 - D(G(\mathbf{z})))$

→ **D가 가짜를 찌처럼 보도록 만들자**

! But

학습 초기:

G가 아무 의미 없는 엉터리 이미지 생성 → D가 이미지를 쉽게 가짜라고 판단

$D(G(\mathbf{z})) \approx 0$

손실 함수 $\log(1 - D(G(\mathbf{z})))$ 는?

ex: $(G(\mathbf{z}))=0.01$ 이라면 $\log(1 - D(G(\mathbf{z}))) = \log(0.99) \approx -0.010$

이 값은 이미 거의 최소값 근처에 있음

그래서 미분하면 기울기(gradient)가 거의 0에 가까워짐

$$\frac{d}{dG} \log(1 - D(G(\mathbf{z}))) \approx 0$$

G는 학습해야 하는데, 기울기가 너무 작아서 학습이 거의 안 됨

→ 기울기 소실 문제 발생

그래서...

학습 초기에 기울기가 크게 나오도록 손실 함수를 바꿔주자!

바꾼 손실 함수: $\max_G \log D(G(z))$ (= 진짜처럼 보이게 만들기)

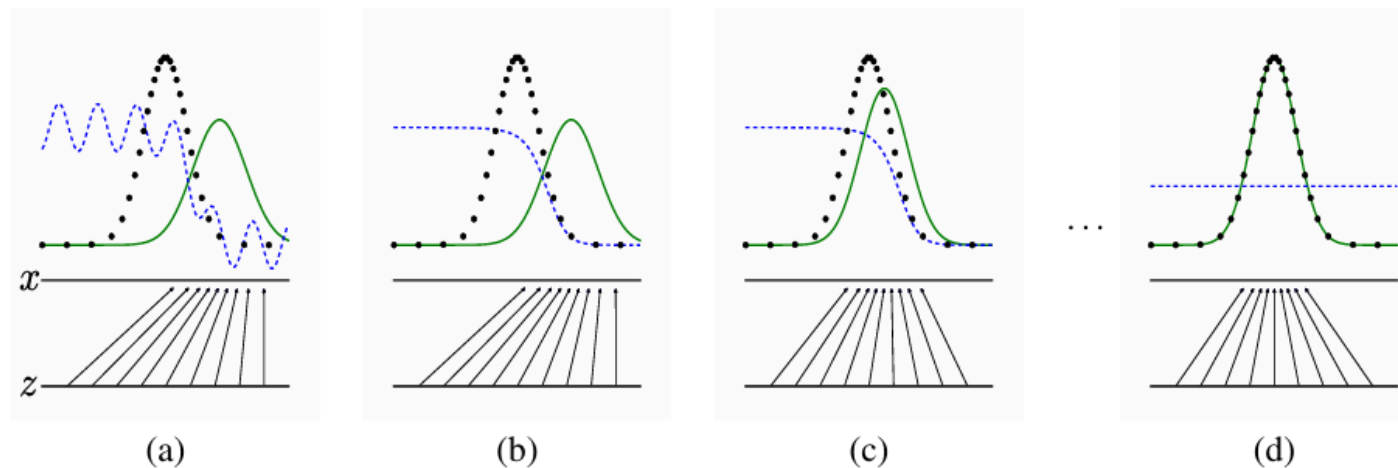
ex: $D(G(z))=0.01$ 이면 $\log D(G(z))=\log(0.01)=-4.6$

→ 손실이 아주 큼 → 미분했을 때 gradient도 큼

→ 학습 잘 됨!

그래서 실제로는 이렇게 바꿈: $\max_G \log D(G(z))$

→ G가 만든 결과가 진짜라고 D가 믿게 만드는 방향, 수식은 다르지만 고정점(fixed point)은 같다



- **(a)** G가 아직 완벽하지 않고, D가 절반쯤 잘 맞히는 상황
- **(b)** D는 점점 p_{data} vs p_g 잘 구분
- **(c)** G는 gradient를 받아서 점점 진짜 데이터 분포 쪽으로 이동
- **(d)** $p_g = p_{\text{data}}$ 가 되면, $D(x)=0.5$ 가 되어 더 이상 개선 불가

4. Theoretical Results

Generator(G)가 만드는 가짜 데이터가 진짜 데이터랑 똑같아질 수 있을까?

4.1 Global Optimality of $p_g = p_{\text{data}}$

Discriminator D를 최적으로 학습했을 때 Generator G가 만들어낸 분포 p_g 가 언제 p_{data} 와 같아지는지 확인

1 Discriminator의 최적값 $D^*(x)$

G가 데이터를 완벽하게 판별할 수 있다고 하면, D는 이렇게 학습: $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$

= 진짜일 확률의 베이즈 계산식과 동일

→ x가 진짜일 확률 = 진짜 분포 / (진짜 + 가짜 분포)

2 D를 최적화한 상태에서 G의 목적 함수

위에 있는 최적 D를 대입하면, G의 실제 손실 함수는:

$$C(G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D^*(x)] + \mathbb{E}_{x \sim p_g} [\log(1 - D^*(x))]$$

$$\rightarrow C(G) = -\log(4) + 2 \cdot \text{JSD}(p_{\text{data}} \parallel p_g)$$

* **JSD (Jensen-Shannon Divergence):** 두 분포 사이의 거리 측도, 항상 0 이상

3 언제 최솟값이 되냐?

$JSD(p_{data}, p_g) = 0 \Rightarrow p_g = p_{data}$

$p_g = p_{data}$ 일 때 손실 C(G)가 최솟값 $-\log 4$ 에 도달

4.2 Convergence of Algorithm 1

실제로 GAN 학습 알고리즘이 $p_g = p_{data}$ 에 도달할 수 있을까?

- G와 D가 무한한 능력(capacity)을 가지고 있고
- D는 매 step마다 항상 최적으로 학습되고
- G는 너무 급격하게 변하지 않고 천천히 업데이트된다면 가능

왜?

G의 손실은 JSD 기반이므로, 매 step마다 **JSD가 줄어들도록** 학습된다면, 결국 $p_g = p_{data}$ 에 도달

핵심 질문	답변
GAN이 진짜 분포 복제 가능?	수학적으로 Yes
어떤 조건에서?	G, D가 충분히 강력하고, D는 최적으로 학습되고, G는 천천히 움직일 때
손실 함수의 의미는?	JSD 기반으로 두 분포 간의 거리 측정
최적값은 언제?	$p_g = p_{data}$ 일 때 도달

5. Experiments

데이터셋

1. **MNIST:** 손글씨 숫자 이미지 (0~9, 28x28)
2. **TFD (Toronto Face Dataset):** 얼굴 이미지 데이터셋
3. **CIFAR-10:** 32x32 크기의 자연 이미지 (비행기, 자동차, 동물 등 10개 클래스)

모델 구조

- **Generator (G)**
 - 활성화함수: ReLU, sigmoid 혼용
 - 입력: 정규분포에서 뽑은 노이즈 z
- **Discriminator (D)**
 - 활성화함수: Maxout
 - 드롭아웃 적용

성능 측정

Parzen window estimation 사용해서 log-likelihood 근사

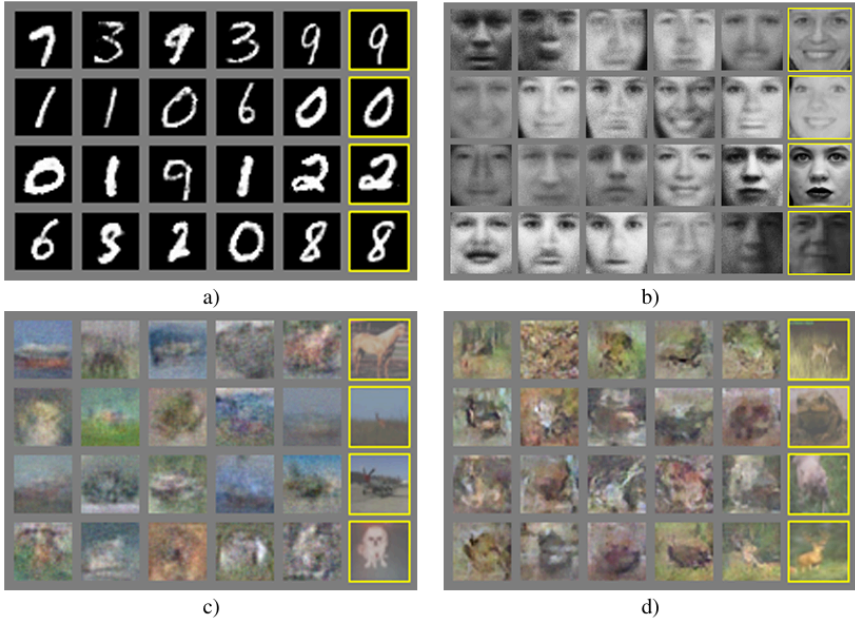
* Parzen window: G가 만든 샘플 기반으로 가우시안 커널로 분포를 근사한 뒤, 진짜 데이터 x가 거기서 나올 확률을 계산하는 방식

실험 결과 (Table 1)

Model	MNIST (↑)	TFD (↑)
DBN (Deep Belief Net)	138 ± 2	1909 ± 66
Stacked CAE	121 ± 1.6	2110 ± 50
Deep GSN	214 ± 1.1	1890 ± 29
Adversarial Nets	225 ± 2	2057 ± 26

→ MNIST에서 가장 높은 성능을 기록, 전반적으로 상위권 성능

시각적 결과



- a) MNIST 샘플
 - b) TFD 얼굴
 - c) CIFAR-10 (fully connected G & D)
 - d) CIFAR-10 (conv & deconv 구조)
- GAN이 진짜처럼 보이는 이미지를 잘 만들어냄



z 공간에서 두 점을 잇는 선형 보간 (interpolation)을 하면 G는 자연스럽게 변하는 이미지 시퀀스 출력

6. Advantages and Disadvantages

구분	장점	단점
확률 계산	안 해도 됨 → 단순함	확률 없어서 평가 어려움
샘플링	빠르고 깔끔함	없음
학습 방식	백프로퍼게이션만으로 충분	D와 G의 균형 잡기가 어려움
구조 자유도	매우 높음	과적합이나 mode collapse 위험 있음